Mechanism for Long Context for text-based LLMs

Aditya Vishwakarma

Abstract

*The paper describes a novel **Divided Context Architecture (DCA)** for Large Language Models (LLMs) that aims to increase context length without proportionally increasing computational demand. The core innovation lies in modifying the **attention mechanism** through the introduction of **vertical retention vectors** and a new **LoTA (Least of Them All)** normalisation function, which performs global score normalisation. The architecture is composed of stacked **"Blocks"** with each block containing **"Parallel"** and **"Partial Attention"** units that share information via these retention vectors, facilitating both horizontal and vertical information flow.*

Introduction

Inspired from the transformer introduced by Vaswani et. Al., DCA introduces new modification, weights and vectors for retaining and increasing context. This is very specific modification to transformer structure where multiple such transformers (block in this context) are stacked over each other and the information is shared to next transformer via vertical retention vectors. The major change is in attention mechanism with new MLP for vertical retention vectors and also vertical weight matrices. For feed forward, there are two MLPs, first one for horizontal and second one is vertical MLP, for horizontal and vertical retention vectors, where horizontal vertical vector $\mathbf{E_H}$ which is single vector with dimension equal to dimension of tokens embeddings, while $\mathbf{E_V}$ is matrix with size as context-window times dimension of token embeddings. For attention score normalisation, the new function is **LoTA**, an acronym for **Least of Them ALL**. This is done for global score normalisation, rather than row-wise SoftMax.

Structure

A matrix of attention mechanism is called as Block, with columns called as **PARALLEL** and rows are **PARTIAL ATTENTION (PA).** Multiple blocks are called as transformer. Context window of single block is called as **LOCAL CONTEXT (C_L).** The whole transformer has **FULL CONTEXT (C_F).** The local context is equal for all blocks in DCA. The Dimensions of components are as follows:

- Context Window = $\mathbf{C_W}$, Local Context = $\mathbf{C_L}$, Full Context = $\mathbf{C_F}$
- Number of Blocks: $\mathbf{N_B}$, Number of PARALLELs: $\mathbf{N_P}$, Number of PAs: $\mathbf{N_{PA}}$
- Number of Attention units per PA = $\mathbf{N_A}$
- Dimension of Embedding Vector: $\mathbf{D_E}$
- Total Vocabulary of tokens: **Vocab**

- Dimension of De-Embedding: $\mathbf{D_D}$ ($D_D = D_E \times N_{PA}$)
- Feature Dimensions: $\mathbf{D_F}$
- Dimension of Input and output vectors of MLP: $\mathbf{D_E}$
- MLP weights matrix: $\mathbf{D_E} \times \mathbf{D_E}$
- Number of Layers of Weights Matrix of MLP: $\mathbf{n_L}$
- FULL CONTEXT: $\mathbf{C_F} = N_B \times C_L$

Attention Unit

A single attention unit or attention is the basic unit of the transformer, with matrix for scaled dot product, matrices for Queries and Keys, Horizontal Projection Matrix for token prediction and Vertical Projection Matrix for vertical retention vectors. One MLP for horizontal flow and second for vertical. These mechanism work in parallel, column-wise through the block. These matrices are used for training and capture all nuances, no matter how common and niche they can be, but for inference use of caches obtained via trained multiplied vector.

Block

A single block is similar to a transformer (Vaswani et. Al.), a 2-dimensional arrangement of attention units, with each unit sharing EH to next one in same PA and EVs to same subsequent unit of next block. The forward propagation and backward propagation i.e., ForProp and BackProp, are executed in parallel through column-based execution. Block also has a common collection of all the EVs from each attention unit and a separate token embedding matrix to hold tokens specific to each block. Columns are referred as **PARALLEL** since they are executed in parallel, while the end head's EH is used for final EH vector (which is either sum of concatenation of all end head's EH of each row), hence they are called **PARTIAL ATTENTION**. Hence the block can be referred as **COMPLETE ATTENTION**.

Transformer

Transformer in DCA is stacked arrangement of Blocks. Here all processes run block-wise. In training, ForProp works on single block, but for backprop, starting from the current block up to first block, process is done. This to correct the vertical retention as much as possible. In inference, only one block is defined, and used also, when context for block is reached, the EVs of same block are used for queries and embeddings from previous context for keys. In inference, Caches are used rather than trained weights. These caches are obtained by multiplying trained weight matrices. Since, the total context of transformer is Local context times number of blocks, transformer is also referred as **FULL CONTEXT**.

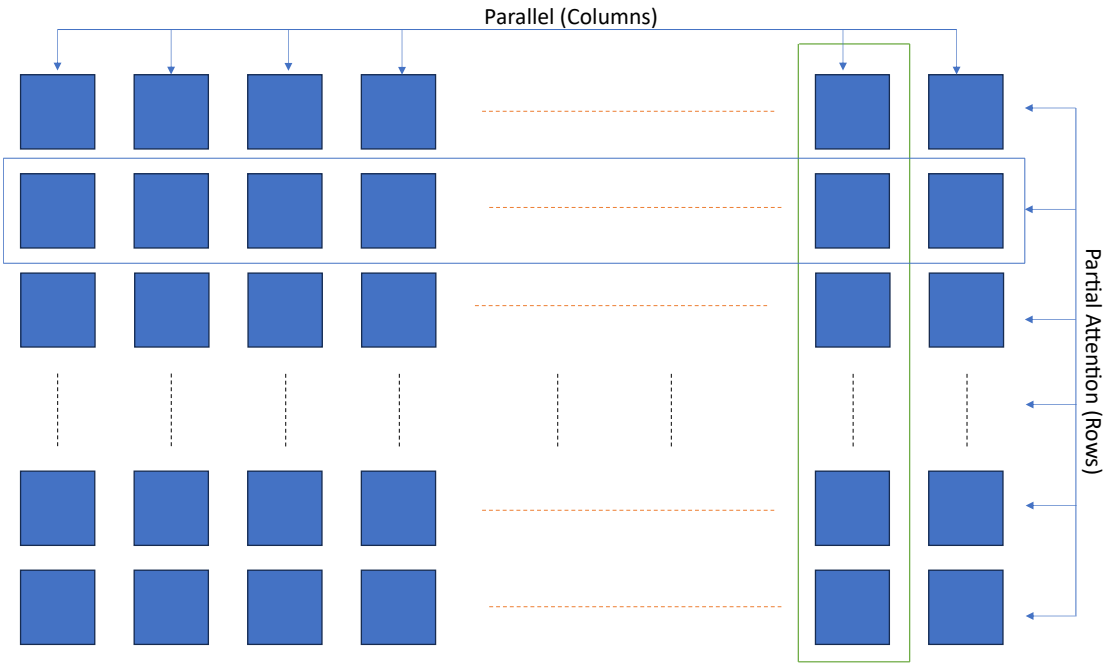

Fig. 1: Single DCA Block

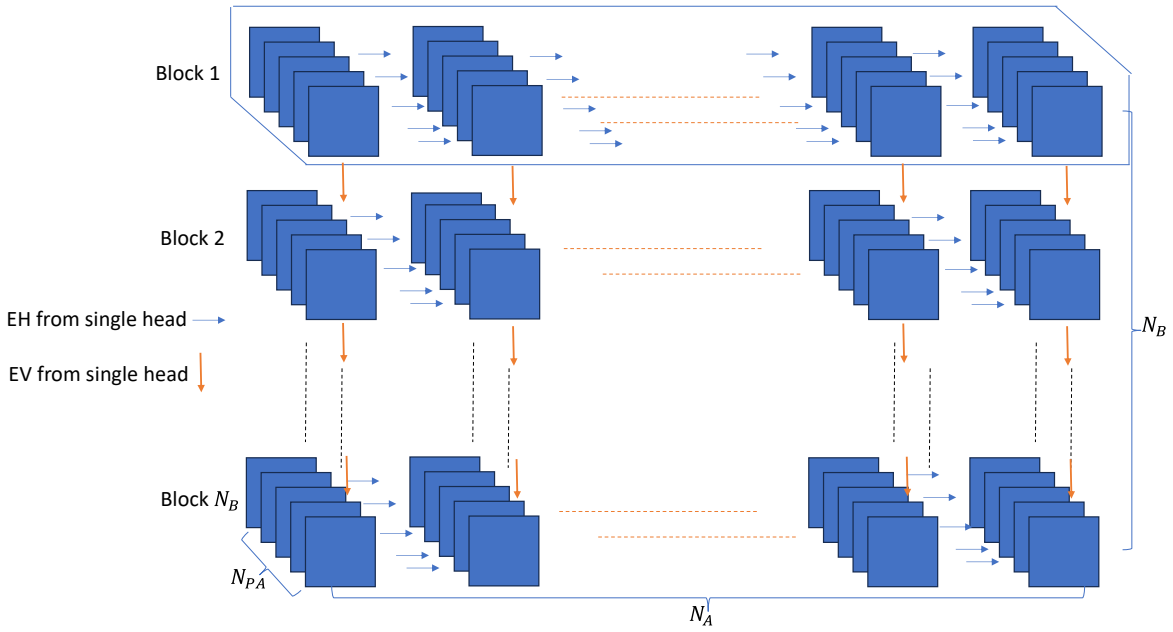

Fig. 2: DCA Transformer

LOTA

Least of Them All (LOTA) is the method to normalise the values of set via most minimum value from the set and then calculate sum of these new values and divide each of them by this sum. Consider M to be a matrix of values, M_{min} is the minimum value out of all. Each value of the grid is subtracted by this minimum value and the sum **s** is calculated from these new values, which are all positive. These positive values are fraction of this sum s and hence when divided by s, what we get is globally normalised score, fractions of these values w.r.t **s**.

Let $S = [S_1, S_2, S_3, S_4, S_5, \cdots, S_m], here\ S_i \in \mathbb{R}$

1. Compute the minimum: $M_{min} = \min(S_1, S_2, S_3, S_4, S_5, \cdots, S_m)$
2. Shift by absolute minimum: $S_i' = S_i + |M_{min}|$
3. Compute Sum: $s = \sum S_i'$
4. Normalise: $P_i = {S_i'}/{S}$

LOTA vs SoftMax

While SoftMax exponentially compresses negative values toward zero (effectively erasing them), LoTA preserves the relative relationships between all scores by shifting them into positive space through global normalization. This distinction is particularly important for long context processing, where subtle negative relationships between distant tokens might carry meaningful information that traditional SoftMax would eliminate. The common theme in these two are division by total sum of new values. While the new values are obtained via subtraction by minimum in LoTA or e^x in SoftMax, the range of these two functions are completely different. Simple comparison lies in the range of functions w.r.t. their domain. The effect these functions have specifically on the negative and positive values respectively. For $\boldsymbol{e^x}$, in domain (-∞, 0) the range is [0, 1) while the simple function like $\boldsymbol{x + |x_{min}|}$ which is linear shift, only increases the value making the minimum value 0 and all the values of the set become positive.

Key Differences

1. **Handling of Negative Values**:
- **SoftMax**: Exponentially compresses negative values toward zero (ex→0 $\boldsymbol{e^x} \to 0$ as $\boldsymbol{x} \to -\infty$. This means negative scores contribute negligibly to the final distribution, potentially losing information encoded in negative relationships.
- **LoTA**: Applies a linear shift by adding $|\boldsymbol{x_{min}}|$, making the smallest value zero and preserving the relative differences between all scores, including negative ones. This ensures that negative relationships (e.g., between distant tokens in a sequence) are not erased.
2. **Range and Transformation**:
- **SoftMax**: Maps the input domain $(-\infty, \infty)$ to the range (0,1), with a strong non-linear effect. Large positive scores dominate due to the exponential function, and small or negative scores are suppressed.
- **LoTA**: Maps the input domain $(-\infty, \infty)$ to [0,1) through a linear shift and normalization. The transformation is linear, preserving the proportional spacing between scores.
3. **Effect on Score Distribution**:
- **SoftMax**: Amplifies differences between large scores due to the exponential function, which can lead to a "winner-takes-all" effect where the largest score dominates the probability distribution.

- **LoTA**: Maintains the relative differences between scores, ensuring that the normalized values reflect the original score distribution more faithfully.

4. **Numerical Stability**:

- **SoftMax**: Can suffer from numerical instability for very large or very small scores due to the exponential function. Techniques like subtracting the maximum score before exponentiation are often used to mitigate this.
- **LoTA**: More numerically stable since it only involves subtraction and division. However, if the sum s is very small (e.g., if all scores are close to the minimum), division by s could amplify noise, though can be rare in practice.

Attention Mechanism

Here are the elements of process of DCA attention mechanism:

- T: Selected Embeddings for prompts and response tokens ($\mathbf{D_E}$)
- E: Embedding matrix ($\mathbf{D_E}$ **x vocab**)
- D: De-embedding matrix ($\mathbf{D_D}$ **x vocab**)
- Q: Query vectors **($\mathbf{C_L}$ x $\mathbf{C_L}$)**
- K: Key vectors **($\mathbf{C_L}$ x $\mathbf{C_L}$)**
- Training Only Weights Matrices:
  - Query weights: W_Q **($\mathbf{D_E}$ x $\mathbf{C_L}$)**
  - Key Weights: W_K **($\mathbf{D_E}$ x $\mathbf{C_L}$)**
  - Horizontal Projection Weights: W_H **($\mathbf{C_L}$ x $\mathbf{D_E}$)**
  - Vertical Projection Weights: W_V **($\mathbf{C_L}$ x $\mathbf{D_E}$)**
- Horizontal and Vertical MLP: $M_H(x)$ and $M_V(x)$
- MLP input: M_{hin} and M_{vin} ($\mathbf{D_E}$)
- MLP output: M_{hout} and M_{vout} ($\mathbf{D_E}$)
- i^{th} weight layer of MLP: $M_{Hi}(x)$ and $M_{Vi}(x)$ ($\mathbf{D_E}$ x $\mathbf{D_E}$)
- Compressed Weights Caches (inference only): W_{QK}, $W_{QV,}$ W_{KH}
- KdotQ: Attention score obtained by Query and Key scaled dot product (without normalisation) **($\mathbf{C_L}$ x $\mathbf{C_L}$)**
- S: Normalised Attention Score **($\mathbf{C_L}$ x $\mathbf{C_L}$)**
- h and v: accumulated horizontal and vertical retention vectors ($\mathbf{D_E}$)
- E_H: horizontal retention vector ($\mathbf{D_E}$)
- H_i: output of i^{th} partial attention of block ($\mathbf{D_E}$)
- H_i^k: Attention output of i^{th} head of k^{th} column ($\mathbf{D_E}$)
- H_o: Concatenated output of block ($\mathbf{D_D}$)
- Z: Raw logits ($= H_o \times D$) **($\mathbf{D_D}$ x vocab)**
- P: $\sigma(Z) =$ Activated Logits as prediction **(vocab)**
- O_{HE}: One Hot Encoding as expected prediction with 1 at location of index of expected token with others as 0s **(vocab)**

- E_V: vertical retention vectors (matrix) **(D_E x C_L)**
- L: Loss obtained from Binary cross entropy BCE(P, O_{HE}) **(vocab)**
- η: Learning Rate
- f: Fraction for vertical MLP gradient ($\boldsymbol{f \leq \eta}$)

ForProp

For first block, the ForProp uses embeddings, for other blocks, it uses EVs from previous blocks and token embeddings used in previous blocks. As provided for attention mechanism, the ForProp process is similar for first and other blocks. It starts from embeddings (and EVs in next block), then Queries and Keys are obtained, dot product is calculated, current tokens, and then scaled dot product vectors are calculated, for both horizontal retention for token prediction and vertical retention for context retention in next block. Then inputs are provided to MLPs, and the output of MLPs are logits which are activated, and then used to update the EH and EV. Updated EH is passed to next unit, and this update continues till last unit is reached. As Local context is reached, EV keeps on updating.

Once, Local Context is reached, the process shifts to next block and uses vertical retention vectors from previous block to calculate queries. Tokens of previous Local context are used for key calculations. Compared to trained matrices, caches can be calculated after training is done. Caches reduce parameters and compute. While trainable matrices can hold more parameters and need more compute, in training, they can hold more nuances and features. This allows, to train model on larger data, extract more features and in inference, use the low parameter caches without losing features.

$$\boldsymbol{KdotQ} = \frac{\boldsymbol{Q} \times \boldsymbol{K^T}}{\sqrt{\boldsymbol{D_E}}}, \qquad Q = T \times W_Q, \qquad K = T \times W_K$$

For (i, j)th value from scaled dot product, these expressions can be given as

$$\boldsymbol{KdotQ(i,j)} = \frac{\boldsymbol{Q(i).K(j)}}{\sqrt{\boldsymbol{D_E}}}, \qquad Q(i) = T(i) \times W_Q, \qquad K(i) = T(i) \times W_K$$

T(i) = ith token embedding vector.

Q(i) and K(i) are query and key vector of T(i).

$$KdotQ(i,j) = \frac{Q(i) \times K(j)^T}{\sqrt{D_E}} = \frac{[T(i) \times W_Q] \times [T(i) \times W_K]^T}{\sqrt{D_E}}$$

$$\because (\boldsymbol{A} \times \boldsymbol{B})^{\boldsymbol{T}} = \boldsymbol{B^T} \times \boldsymbol{A^T}$$

$$KdotQ(i,j) = \frac{[T(i) \times W_Q] \times [W_K{}^T \times T(j)^T]}{\sqrt{D_E}} = \frac{T(i) \times (\boldsymbol{W_Q} \times \boldsymbol{W_K{}^T}) \times T(j)^T}{\sqrt{D_E}}$$

$$\therefore \boldsymbol{W_{QK} = Q_W \times {K_W}^T}$$

The scaled dot products are normalised to give us attention scores in the form of probability distribution.

$$\boldsymbol{LoTA(i,\ j) = \frac{KdotQ(i,\ j) - KdotQ_{min}}{s} = \frac{KdotQ(i,\ j) + |KdotQ_{min}|}{s}}$$

$$where: s = \sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} (KdotQ(i,\ j) + |KdotQ_{min}|) \right.$$

Now, for upgrading vectors dh and dv via normalised scores for Keys and Queries:

$$h = \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LoTA(i,\ j) \right) . K(i) \right] \times W_H$$

$$= \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LoTA(i,j) \right) . T(i) \right] \times W_K \times W_H$$

$$v = \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LoTA(j,\ i) \right) . Q(i) \right] \times W_V$$

$$= \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LoTA(j,i) \right) . T(i) \right] \times W_Q \times W_V$$

For inference, they take the form of:

$$\boldsymbol{K_W \times H_W = W_{KH}, \qquad Q_W \times V_W = W_{QV}}$$

This modification done here provides two types of weights matrices, first one is trainable and second one is compressed weights. The four trainable weights matrices are $\boldsymbol{W_Q, W_K, W_H}$ and $\boldsymbol{W_V}$ and compressed weights are referred as caches, $\boldsymbol{W_{KH}}$, $\boldsymbol{W_{QV}}$ and $\boldsymbol{W_{QK}}$ respectively. These compressed weights are used for inference.

Limit is used to describe specific value, which incudes, total tokens of prompts and previous response and also current response, or in simple terms, current token count. The activation function used is Sigmoid. **isSelf** is a Boolean value, when 0 means cross-attention and 1 means self-attention, combined with ternary operator, for setting upper limit for maximum values to be included in sum for cross and self-attention. For isSelf = 1, the KdotQ has lower triangle values while for isSelf = 0, the grid has values in all places of limit x limit.

$$sigmoid\ activation\ function: \sigma(x) = \frac{1}{1 + e^{-x}}$$

For retention vectors:

$$M_{Hin} = E_H + h, \qquad M_{Vin} = \sum E_V(i) + v$$

$$\boldsymbol{E_H} \mathbf{+=} \ \boldsymbol{M_{H(out)}}, \qquad \boldsymbol{E_V(i)} \mathbf{+=} \ \boldsymbol{M_{V(out)}}$$

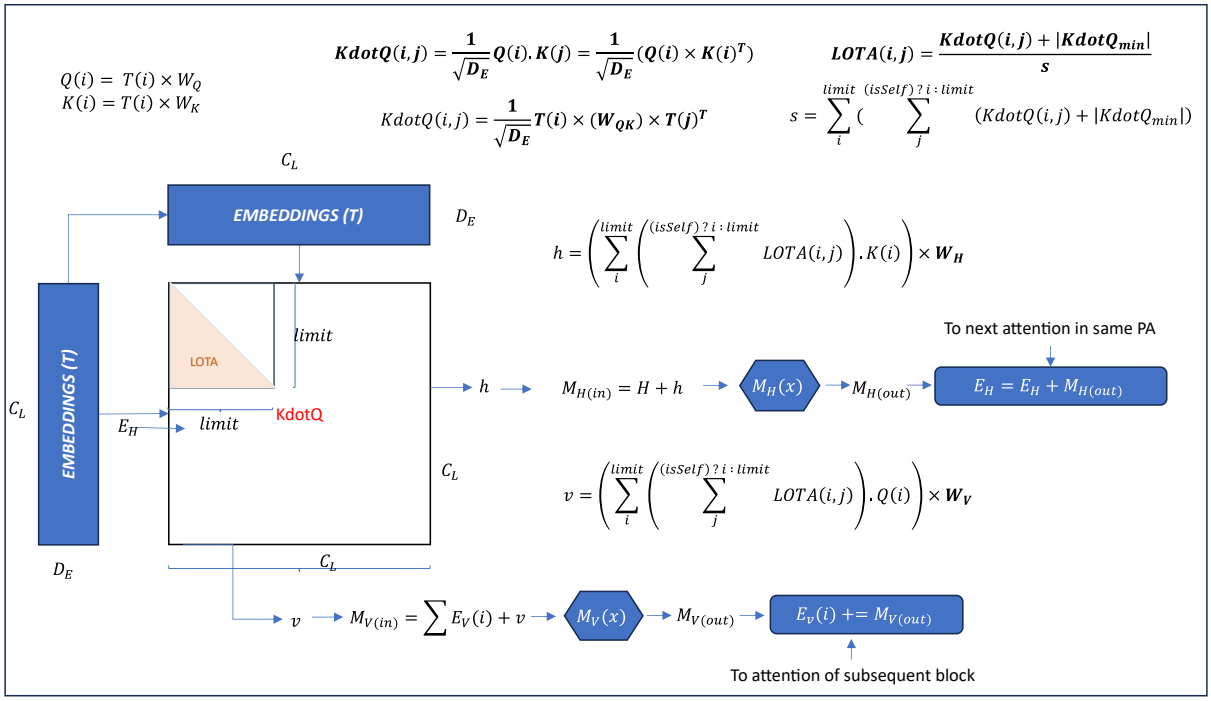

Fig. 3: For attention units in first block

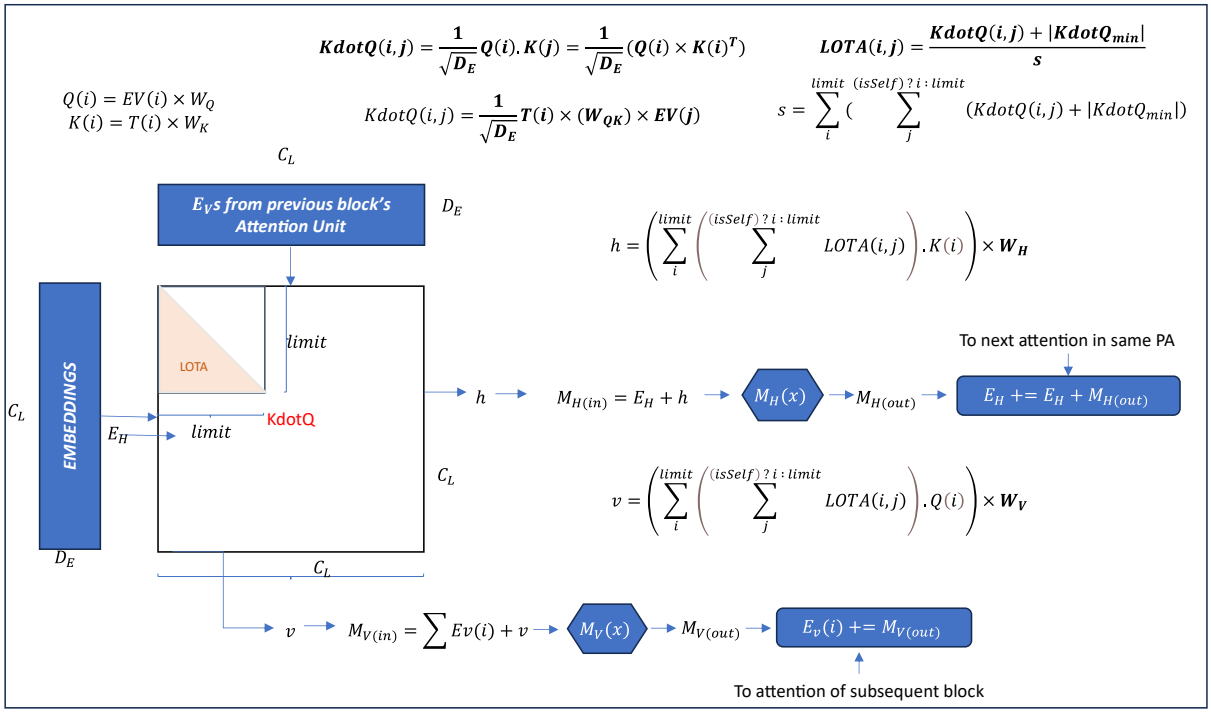

Fig. 4: For attention units of 2nd to last blocks

$$H_o = concat(H_1^n, H_2^n, \dots, \dots, H_m^n), \qquad size = d = D_D$$

$$m = N_{PA}$$

$$Z(i) = \sum_{k=1}^{d} H_o(k).D(k,i)$$

$$Z = H_o \times D$$

BackProp

Suppose n^{th} block is the current block, where the process is taking place. The backprop starts from expected token, from last column to first, and this block to first block. In the same block, backprop works on the basis of EH and for other upward blocks, $(i - 1)^{th}$ to 1^{st}, based on the EVs of the i^{th} to 2^{nd} block. Hence, there are two types of backpropagations. This can be further be classified for four cases based on how EH and EVs are updated based on the head location and block.

Following are Sigmoid and LOTA derivatives, which are necessary for calculating gradients.

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

$$LoTA'(i,j) = \frac{1 - LOTA(i,j)}{s}$$

And the Binary cross entropy loss function L is given by

$$L = -\sum_{i=1}^{vocab} (O_{HE} \log_{10} P - (1 - O_{HE}) \log_{10}(1 - P))$$

By chain rule, the gradient of Loss w.r.t. raw logits are

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial P} \times \frac{\partial P}{\partial Z} = P - T$$

For D, the gradient is obtained as:

$$\frac{\partial L}{\partial D} = \frac{\partial L}{\partial Z} \times \frac{\partial Z}{\partial D} = H_o^T \frac{\partial L}{\partial Z}$$

$$\boldsymbol{D \leftarrow D - \eta \frac{\partial L}{\partial D} = D - \eta H_o^T \frac{\partial L}{\partial Z}}$$

For H_o, the gradient is obtained as:

$$\frac{\partial L}{\partial H_o} = \frac{\partial L}{\partial Z} \times \frac{\partial Z}{\partial H_o} = \frac{\partial L}{\partial Z} D^T$$

$$\boldsymbol{H_o \leftarrow H_o - \eta \frac{\partial L}{\partial H_o} = H_o - \eta \frac{\partial L}{\partial Z} D^T}$$

The transpose of the specific objects is taken to accommodate the dimensions of the other objects and them being matrix or vector. From here, the backprop starts in two directions, for horizontal with token prediction and vertical for vertical

retention vectors. This new H_o is used for backpropagation in horizontal direction, and vertical backpropagation happens in simultaneously.

Backprop in First Block

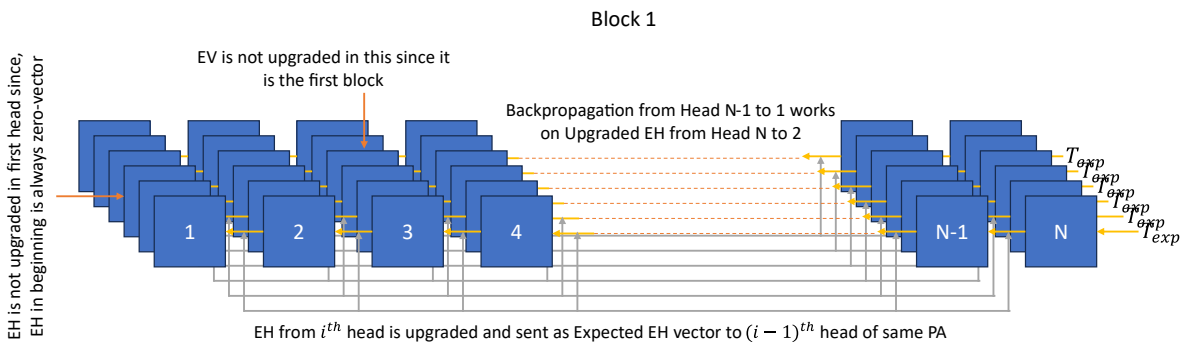

Fig. 5: Backprop in First Block for Token Prediction

Updated H_o is treated as expected vector for the last column and here it will be compared with original attention-wise outputs, and parts of H_o for backpropagation. Following is the output from final column by residual connection of output of MLP $M_H(x)$ and E_H of previous head of same partial attention.

$$H_i^m = M_{hout}^m + E_H^{m-1}$$

$$\frac{\partial L}{\partial M_{hout}^m} = \frac{\partial L}{\partial H_i^m} \cdot \frac{\partial H_i^m}{\partial M_{hout}^m} = \frac{\partial L}{\partial H_i^m}$$

$$\left.\frac{\partial L}{\partial \boldsymbol{E_H^{m-1}}}\right|_{\boldsymbol{post}} = \frac{\partial L}{\partial H_i^m} \cdot \frac{\partial H_i^m}{\partial E_H^{m-1}} = \frac{\partial L}{\partial H_i^m}$$

In case of $M_V(x)$, a fraction of gradient of M_{hout}^m is passed to vertical retention vectors for future predictions:

$$\frac{\partial L}{\partial M_{vout}^m} = f \cdot \frac{\partial L}{\partial M_{hout}^m}$$

$$\boldsymbol{E_V^m}(i) = M_{hout}^m + \boldsymbol{E_V^{m-1}}(i)$$

$$\left.\frac{\partial \boldsymbol{L}}{\partial \boldsymbol{E_H^{m-1}(i)}}\right|_{\boldsymbol{post}} = f \cdot \frac{\partial L}{\partial M_{hout}^m}$$

The gradient obtained from H_o is provided to both $M_H(x)$ and E_H^{m-1}. The gradient for M_{hout}^m is passed to the mlp and gradients for each layer are calculated along with the gradient for input M_{hin}^m.

$$x = M_{hin}^m, \qquad M_{hout}^m = M_H^m(x) = M_{Hl}$$

$$\frac{\partial M_{H(i+1)}}{\partial M_{Hi}} = \frac{\partial}{\partial M_{Hi}}\left(\sigma(M_{Hi}W_{i+1} + b_{i+1})\right) = M_{Hi}(1 - M_{Hi})W_{i+1}^T$$

$$\frac{\partial L}{\partial M_{Hi}} = \frac{\partial L}{\partial M_{H(i+1)}} . \frac{\partial M_{H(i+1)}}{\partial M_{Hi}} = \frac{\partial L}{\partial M_{H(i+1)}} . M_{Hi}(1 - M_{Hi}) W_{(i+1)}^{T}$$

$$\frac{\partial L}{\partial M_{hin}^{m}} = \frac{\partial L}{\partial M_{H1}} . \frac{\partial M_{H1}}{\partial M_{hin}^{m}} = \frac{\partial L}{\partial M_{H1}} . M_{hin}^{m}(1 - M_{hin}^{m}) W_{1}^{T}$$

The input to MLP is obtained by residual connection and hence the gradient obtained is split equally to both attention output 'h' and E_H.

$$M_{hin}^{m} = h + E_{H}^{m-1}$$

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial M_{hin}^{m}} . \frac{\partial M_{hin}^{m}}{\partial h} == \frac{\partial L}{\partial M_{hin}^{m}}$$

$$\left. \frac{\boldsymbol{\partial L}}{\boldsymbol{\partial E_{H}^{m-1}}} \right|_{\boldsymbol{pre}} = \frac{\partial L}{\partial M_{hin}^{m}} . \frac{\partial M_{hin}^{m}}{\partial E_{H}^{m-1}} = \frac{\partial L}{\partial M_{hin}^{m}}$$

Similarly, for $M_V(x)$:

$$\frac{\partial M_{V(i+1)}}{\partial M_{Vi}} = \frac{\partial}{\partial M_{Vi}} \left(\sigma(M_{Vi} W_{i+1} + b_{i+1}) \right) = M_{Vi}(1 - M_{Vi}) W_{i+1}^{T}$$

$$\frac{\partial L}{\partial M_{Vi}} = \frac{\partial L}{\partial M_{V(i+1)}} . \frac{\partial M_{H(i+1)}}{\partial M_{Vi}} = \frac{\partial L}{\partial M_{V(i+1)}} . M_{Vi}(1 - M_{Vi}) W_{(i+1)}^{T}$$

$$\frac{\partial L}{\partial M_{vin}^{m}} = \frac{\partial L}{\partial M_{V1}} . \frac{\partial M_{H1}}{\partial M_{vin}^{m}} = \frac{\partial L}{\partial M_{V1}} . M_{vin}^{m}(1 - M_{vin}^{m}) W_{1}^{T}$$

$$M_{vin}^{m} = h + \sum_{i} E_{V}^{m-1}(i)$$

$$\frac{\partial L}{\partial v} = \frac{\partial L}{\partial M_{vin}^{m}} . \frac{\partial M_{vin}^{m}}{\partial v} == \frac{\partial L}{\partial M_{vin}^{m}}$$

$$\left. \frac{\partial L}{\partial E_{V}^{m-1}(i)} \right|_{pre} = \frac{\partial L}{\partial M_{vin}^{m}} . \frac{\partial M_{vin}^{m}}{\partial E_{V}^{m-1}} = \frac{\partial L}{\partial M_{vin}^{m}}$$

From h and v:

$$\frac{\partial L}{\partial W_{H}} = \frac{\partial L}{\partial h} . \frac{\partial h}{\partial W_{H}} = \left(\frac{\partial L}{\partial h} \right)^{T} \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LOTA(i, j) \right) . K(i) \right]$$

$$\frac{\partial L}{\partial W_V} = \frac{\partial L}{\partial v}.\frac{\partial v}{\partial W_V} = \left(\frac{\partial L}{\partial v}\right)^T \left[\sum_{i=1}^{limit}\left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LOTA(j,i)\right).Q(i)\right]$$

$$\boldsymbol{W_H \leftarrow W_H - \eta\frac{\partial L}{\partial W_H}}, \qquad \boldsymbol{W_V \leftarrow W_V - \eta\frac{\partial L}{\partial W_V}}$$

$$\left.\frac{\partial L}{\partial K}\right|_h = \frac{\partial L}{\partial h}.\frac{\partial h}{\partial K} = S.\left[\frac{\partial L}{\partial h} \times W_H^T\right]$$

$$\left.\frac{\partial L}{\partial Q}\right|_v = \frac{\partial L}{\partial v}.\frac{\partial v}{\partial Q} = S.\left[\frac{\partial L}{\partial v} \times W_V^T\right]$$

$$\left.\frac{\partial L}{\partial S}\right|_h = \frac{\partial L}{\partial h}.\frac{\partial h}{\partial S} = N_T K\left[\frac{\partial L}{\partial h} \times W_H^T\right], \qquad \left.\frac{\partial L}{\partial S}\right|_v = \frac{\partial L}{\partial v}.\frac{\partial v}{\partial S} = N_T K\left[\frac{\partial L}{\partial v} \times W_V^T\right]$$

$$\boldsymbol{\frac{\partial L}{\partial S}} = N_T\left(K\left[\frac{\partial L}{\partial h} \times W_H^T\right] + K\left[\frac{\partial L}{\partial v} \times W_V^T\right]\right)$$

$$\left.\frac{\partial L}{\partial T}\right|_h = \frac{\partial L}{\partial h}\frac{\partial h}{\partial T} = \frac{\partial L}{\partial h}(SW_K^T)W_H, \qquad \left.\frac{\partial L}{\partial T}\right|_v = \frac{\partial L}{\partial v}\frac{\partial v}{\partial T} = \frac{\partial L}{\partial v}(SW_Q^T)W_V$$

From attention scores:

$$A(i,j) = KdotQ(i,j) = \frac{Q(i).K(j)}{\sqrt{D_E}} \rightarrow A = KdotQ = \frac{Q \times K^T}{\sqrt{D_E}} \rightarrow S = LOTA(A)$$

$$\boldsymbol{\frac{\partial L}{\partial A}} = \frac{\partial L}{\partial S}.\frac{\partial S}{\partial A} = \frac{\partial L}{\partial S}.\frac{\partial}{\partial A}LOTA(A) = \frac{\partial L}{\partial S}\left(\frac{1 - LOTA(A)}{s}\right)$$

$$\left.\frac{\partial L}{\partial K}\right|_A = \frac{\partial L}{\partial A}\frac{\partial A}{\partial K} = \frac{\partial L}{\partial A}\frac{\partial}{\partial K}\left(\frac{Q \times K^T}{\sqrt{D_E}}\right) = \frac{\partial L}{\partial A}\frac{Q^T}{\sqrt{D_E}}$$

$$\left.\frac{\partial L}{\partial Q}\right|_A = \frac{\partial L}{\partial A}\frac{\partial A}{\partial Q} = \frac{\partial L}{\partial A}\frac{\partial}{\partial Q}\left(\frac{Q \times K^T}{\sqrt{D_E}}\right) = \frac{K}{\sqrt{D_E}}\frac{\partial L}{\partial A}$$

$$\boldsymbol{\frac{\partial L}{\partial K}} = \left.\boldsymbol{\frac{\partial L}{\partial K}}\right|_{\boldsymbol{h}} + \left.\boldsymbol{\frac{\partial L}{\partial K}}\right|_{\boldsymbol{A}}, \qquad \boldsymbol{\frac{\partial L}{\partial Q}} = \left.\boldsymbol{\frac{\partial L}{\partial Q}}\right|_{\boldsymbol{v}} + \left.\boldsymbol{\frac{\partial L}{\partial Q}}\right|_{\boldsymbol{A}}$$

$$\frac{\partial L}{\partial W_K} = \frac{\partial L}{\partial K}\frac{\partial K}{\partial W_K} = \frac{\partial L}{\partial K}\frac{\partial}{\partial W_K}(T \times W_K) = \frac{\partial L}{\partial K}T^T$$

$$\frac{\partial L}{\partial W_Q} = \frac{\partial L}{\partial Q}\frac{\partial Q}{\partial W_K} = \frac{\partial L}{\partial Q}\frac{\partial}{\partial W_K}\left(T \times W_Q\right) = \frac{\partial L}{\partial Q}T^T$$

$$\boldsymbol{W_K \leftarrow W_K - \eta \frac{\partial L}{\partial W_K}}, \qquad \boldsymbol{W_Q \leftarrow W_Q - \eta \frac{\partial L}{\partial W_Q}}$$

$$\left.\frac{\partial L}{\partial T}\right|_K = \frac{\partial L}{\partial K}\frac{\partial K}{\partial T} = \frac{\partial L}{\partial K}\frac{\partial}{\partial T}(T \times W_K) = \frac{\partial L}{\partial K}W_K^T$$

$$\left.\frac{\partial L}{\partial T}\right|_Q = \frac{\partial L}{\partial Q}\frac{\partial Q}{\partial T} = \frac{\partial L}{\partial Q}\frac{\partial}{\partial T}(T \times W_Q) = \frac{\partial L}{\partial Q}W_Q^T$$

$$\boldsymbol{T \leftarrow T - \eta \left[\left.\frac{\partial L}{\partial T}\right|_h + \left.\frac{\partial L}{\partial T}\right|_v + \left.\frac{\partial L}{\partial T}\right|_K + \left.\frac{\partial L}{\partial T}\right|_Q\right]}$$

Backprop in Subsequent Blocks

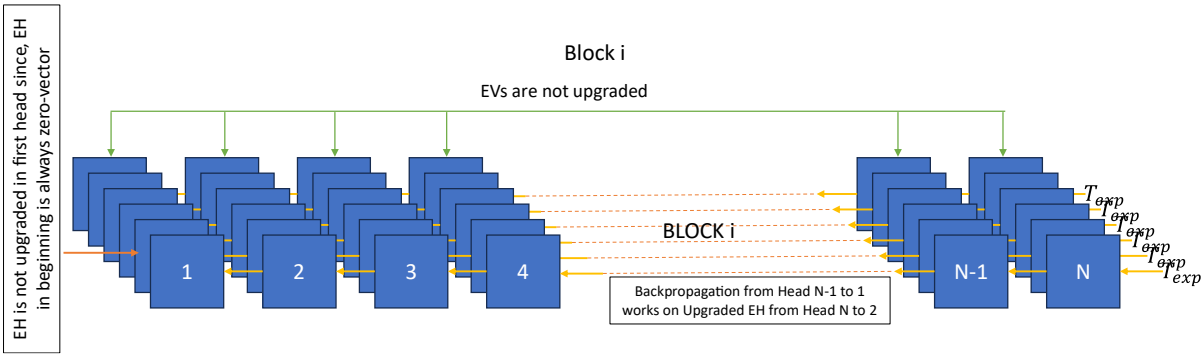

Fig. 6: Backprop for i[th] Block for Token Prediction

For subsequent blocks, the process is all similar, except query weights will be updated based on the Query vectors, and EVs from previous block. Another major change is regarding update of embeddings in use and EVs from previous block won't be updated. Also, Embeddings are not modified, since, it is done only in first block training.

$$\boldsymbol{Q = E_V \times W_Q}$$

$$\left.\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial E_V}}\right|_v = \frac{\partial L}{\partial v}\frac{\partial v}{\partial T} = \frac{\partial L}{\partial v}(SW_Q^T)W_V$$

$$\left.\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial E_V}}\right|_Q = \frac{\partial L}{\partial Q}\frac{\partial Q}{\partial \boldsymbol{E_V}} = \frac{\partial L}{\partial Q}\frac{\partial}{\partial \boldsymbol{E_V}}(\boldsymbol{E_V} \times W_Q) = \frac{\partial L}{\partial Q}W_Q^T$$

$$\frac{\boldsymbol{\partial L}}{\boldsymbol{\partial W_Q}} = \frac{\partial L}{\partial Q}\frac{\partial Q}{\partial W_Q} = \frac{\partial L}{\partial Q}\frac{\partial}{\partial W_Q}(E_V \times W_Q) = \frac{\partial L}{\partial Q}E_V^T$$

If there is need to contextualise embedding at this stage, it can be done via this:

$$\boldsymbol{T \leftarrow T - \eta \left[\left.\frac{\partial L}{\partial T}\right|_h + \left.\frac{\partial L}{\partial T}\right|_v + \left.\frac{\partial L}{\partial T}\right|_K\right]}$$

Training

First block training is specifically utilised to train the embeddings and de-embeddings, along with the block. After full training in local context of first block, all the values of matrices and MLPs are copied to other blocks and then they are trained for full context.

Subsequent blocks or non-first blocks, are trained modify the shared values w.r.t. vertical context, embeddings and de-embeddings. This helps in keeping the flow of model intact and avoid the hallucination from the model. Embeddings and de-embeddings may or may not be updated depending on the design choice.

Inference

With simple linear algebraic transformation, matrix multiplications can be transformed and compute can be reduced. For Inference, caches are used in following manner in place of trained matrices:

$$KdotQ(i,j) = \begin{cases} \dfrac{T(i) \times \boldsymbol{W_{QK}} \times T(j)^T}{\sqrt{D_E}}, & for\ first\ block \\ \dfrac{T(i) \times \boldsymbol{W_{QK}} \times E_V(j)^T}{\sqrt{D_E}}, & for\ other\ blocks \end{cases}$$

$$h = \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LOTA(i,\ j) \right) . T(i) \right] \times \boldsymbol{W_{KH}}$$

$$v = \begin{cases} \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LOTA(j,\ i) \right) . T(i) \right] \times \boldsymbol{W_{QV}}, & for\ first\ block \\ \left[\sum_{i=1}^{limit} \left(\sum_{j=1}^{(isSelf)\ ?\ i\ :\ limit} LOTA(j,\ i) \right) . E_V(i) \right] \times \boldsymbol{W_{QV}}, & for\ other\ blocks \end{cases}$$

Reference

1. Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Polosukhin, Illia. Attention is all you need. 2017. https://doi.org/10.48550/arXiv.1706.03762